

HYPOHEALTH

Sistema Inteligente de Gerenciamento e Dispensação de Medicamentos

Integração IoT + Aplicativo Móvel

Documentação Técnica

Instituição	FATEC Jacareí – Prof. Francisco de Moura
Curso	4º DSM – Desenvolvimento de Software Multiplataforma
Projeto	ABP 2026-1 – Desafio de Inovação em IoT
Equipe	Adson Filho, Andressa Toledo, Carlos Magalhães, Eduardo Henrique e João Pedro Anjos
Organização	HypogramTechnologies
Repositório	github.com/HypogramTechnologies/HypoHealth
Versão	1.0 – Sprint 3
Data	Junho de 2026

1. Introdução

O HypoHealth é um sistema integrado de gerenciamento e dispensação de medicamentos, desenvolvido como projeto ABP (Aprendizagem Baseada em Projetos) do 4º semestre do curso de Desenvolvimento de Software Multiplataforma da FATEC Jacareí, pela equipe Hypogram.

O sistema visa apoiar pacientes — especialmente idosos — no controle rigoroso de horários e dosagens de medicamentos, reduzindo erros de administração e promovendo autonomia. Cuidadores e familiares são notificados remotamente quando o medicamento não for retirado no tempo esperado.

Este documento reúne toda a documentação técnica produzida pela equipe, dividida em seções para melhor compreensão de cada aspecto do projeto: arquitetura, componentes IoT, casos de uso, fluxos de comunicação e cronograma.

1.1 Desafio

O desafio proposto consiste no desenvolvimento de um sistema integrado formado por uma caixa organizadora de medicamentos baseada em IoT, um servidor em nuvem e um aplicativo móvel. O dispositivo deverá armazenar medicamentos em diferentes compartimentos, liberando apenas o compartimento correspondente ao horário programado. Além disso, o sistema emitirá um alerta sonoro no momento correto para lembrar o usuário da administração do medicamento, mantendo os demais compartimentos bloqueados para evitar erros.

1.2 Escopo do Sistema

A solução é composta por três camadas integradas:

- Dispositivo IoT físico: caixa organizadora de medicamentos com compartimentos controlados por motor de passos (via driver ULN2003), sensor PIR e buzzer — gerenciados pelo ESP32.
- Backend em nuvem: API REST (Node.js + TypeScript + Express) responsável por orquestrar a lógica de negócio e persistir os dados.
- Aplicativo Móvel (Expo/React Native): interface para cadastro de medicamentos, monitoramento em tempo real e recebimento de notificações.

1.3 Tecnologias Utilizadas

Camada	Tecnologia / Ferramenta	Finalidade
IoT / Hardware	ESP32	Microcontrolador principal do dispositivo
IoT / Hardware	Driver ULN2003	Driver de controle do motor de passos
IoT / Hardware	Motor de Passos	Rotação e posicionamento dos compartimentos
IoT / Hardware	Sensor PIR	Deteção de presença/retirada do medicamento
IoT / Hardware	Buzzer	Alerta sonoro no horário da dose
IoT / Hardware	Protoboard + Caixa	Estrutura física dos compartimentos
Comunicação	MQTT (HiveMQ Cloud)	Protocolo de mensagens IoT (pub/sub)
Backend	Node.js + Express + TS	Servidor, API REST e lógica de negócio
Backend	Docker	Containerização da aplicação
Mobile	Expo (React Native)	Aplicativo iOS e Android

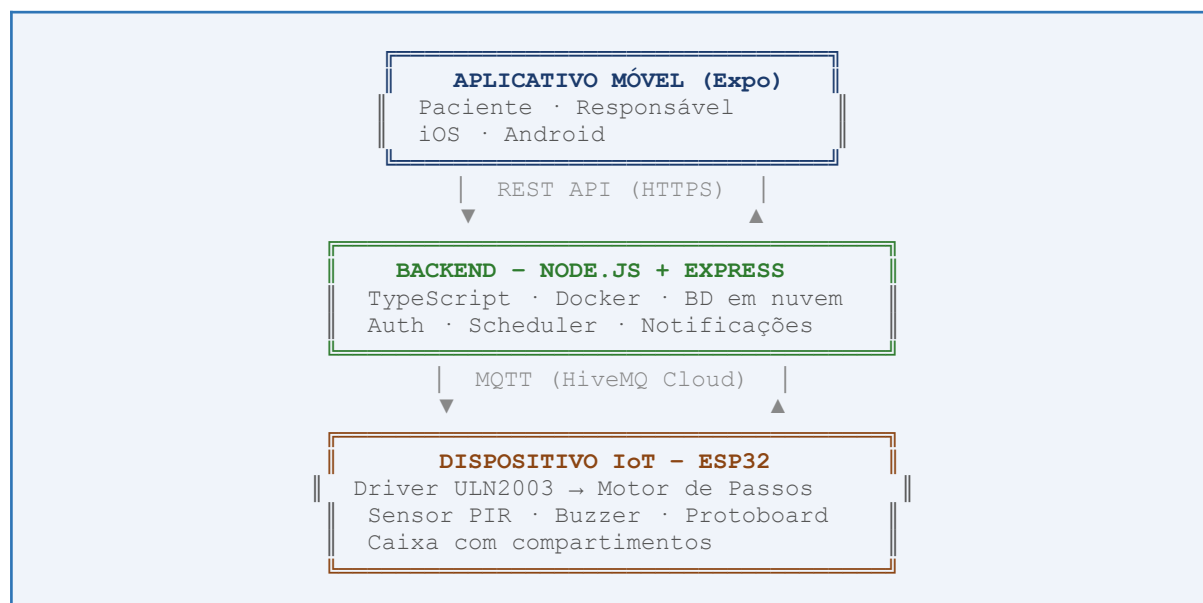
Camada	Tecnologia / Ferramenta	Finalidade
DevOps	GitHub Actions	Pipeline CI/CD automatizado
Design	Figma	Prototipagem de telas
Gestão	Trello	Gerenciamento de tarefas/sprints
IDE	VS Code	Ambiente de desenvolvimento
Versionamento	Git & GitHub	Controle de versão e colaboração

2. Documentação da Arquitetura

2.1 Visão Geral

O HypoHealth segue uma arquitetura distribuída em três camadas (cliente-servidor), com comunicação assíncrona via MQTT entre o dispositivo IoT e o backend, e comunicação síncrona via API REST entre o aplicativo móvel e o backend.

2.2 Diagrama de Componentes



2.3 Descrição das Camadas

2.3.1 Camada IoT (ESP32)

O dispositivo físico é o ponto de interação com o paciente. Seus componentes e responsabilidades são:

- **ESP32:** microcontrolador principal. Conecta ao Wi-Fi, comunica-se com o HiveMQ via MQTT, controla os atuadores e lê os sensores.
- **Driver ULN2003:** módulo responsável pelo controle preciso do motor de passos. Recebe sinais de direção (DIR) e pulso (STEP) do ESP32 e gerencia a corrente para as bobinas do motor, permitindo controle de passo inteiro, meio passo e micropassos.
- **Motor de Passos:** responsável por girar a caixa de compartimentos, posicionando o compartimento correto para abertura no horário programado e bloqueando os demais.
- **Sensor PIR:** detecta presença ou retirada do medicamento no compartimento aberto. Caso não detecte movimento no prazo configurado, reporta o evento 'missed' ao backend via MQTT.
- **Buzzer:** emite sinal sonoro no momento em que a dose deve ser administrada, alertando o paciente.
- **Protoboard:** suporte físico para a montagem e interconexão dos componentes eletrônicos.

2.3.2 Diagrama do Circuito IoT

A figura abaixo apresenta o esquema de montagem do circuito físico do dispositivo HypoHealth, mostrando as conexões entre o ESP32, o driver ULN2003, o motor de passos, o sensor PIR e o buzzer na protoboard:

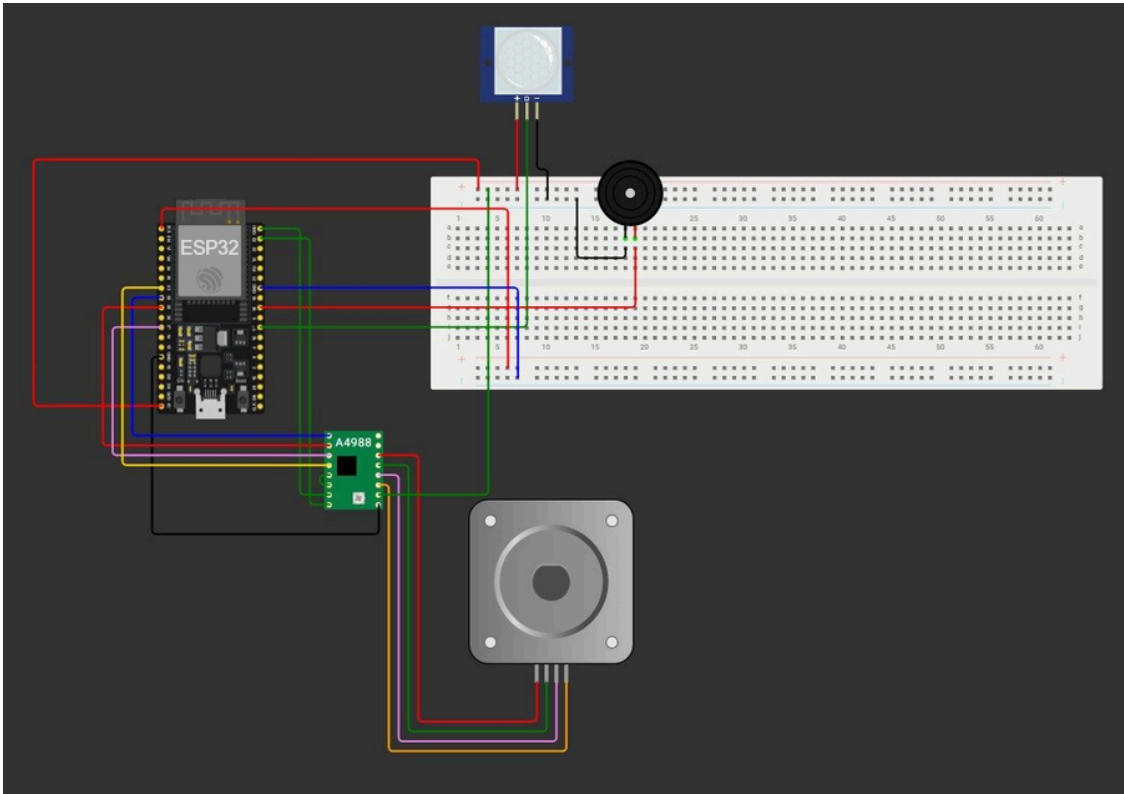


Figura 1 – Diagrama de circuito do dispositivo IoT HypoHealth

2.3.3 Mapeamento de Pinos – ESP32

A tabela abaixo descreve as conexões entre o ESP32 e os demais componentes, conforme o circuito montado:

Pino ESP32	Componente	Função
GPIO (STEP)	Driver ULN2003	Pulso de passo – cada pulso avança o motor 1 passo
GPIO (DIR)	Driver ULN2003	Direção de rotação do motor (HIGH = horário, LOW = anti-horário)
3.3V / GND	Driver ULN2003	Alimentação lógica do driver
VIN / GND	Driver ULN2003	Alimentação de potência para as bobinas do motor (via fonte externa)
A / B / C / D	Motor de Passos	Bobinas do motor – controladas pelo driver ULN2003
GPIO (PIR)	Sensor PIR	Leitura do sinal de detecção de presença (HIGH = movimento detectado)
3.3V / GND	Sensor PIR	Alimentação do módulo PIR
GPIO (BUZZ)	Buzzer	Controle do alerta sonoro (HIGH = ativo)
GND	Buzzer	Referência de terra do buzzer

2.3.4 Sobre o Driver ULN2003

O ULN2003 é um driver de motor de passos bipolar fabricado pela Allegro MicroSystems, amplamente utilizado em projetos embarcados. Suas principais características relevantes para o projeto são:

- Suporta corrente de saída de até 2A por bobina, adequado para o motor utilizado.
- Permite seleção de resolução de passo: passo inteiro, 1/2, 1/4, 1/8 e 1/16 passo, via pinos MS1, MS2 e MS3.
- Possui proteção contra sobrecorrente, subtensão e sobretemperatura.
- Interface simples com o ESP32: apenas dois sinais de controle (STEP e DIR) são suficientes para operação completa.
- Requer ajuste do potenciômetro de corrente (Vref) para calibrar a corrente máxima de acordo com o motor utilizado.

2.3.5 Camada de Comunicação – MQTT (HiveMQ)

A comunicação entre o ESP32 e o backend utiliza o protocolo MQTT, adequado a dispositivos embarcados com recursos limitados. O HiveMQ Cloud atua como broker. Tópicos utilizados:

Tópico MQTT	Direção	Descrição
<code>hypohealth/{deviceId}/schedule</code>	Backend → ESP32	Envia programação de horários e compartimentos
<code>hypohealth/{deviceId}/dispense</code>	Backend → ESP32	Comando para abrir compartimento e acionar buzzer
<code>hypohealth/{deviceId}/taken</code>	ESP32 → Backend	Confirma retirada do medicamento (PIR detectou)
<code>hypohealth/{deviceId}/missed</code>	ESP32 → Backend	Informa que o medicamento não foi retirado no prazo
<code>hypohealth/{deviceId}/status</code>	ESP32 → Backend	Heartbeat periódico do dispositivo (online/offline)

2.3.6 Camada de Backend

O backend (Node.js + Express + TypeScript) é o núcleo de inteligência do sistema. Responsabilidades:

- Autenticação de usuários via JWT.
- Gestão do cadastro de medicamentos e programação de horários.
- Scheduler que aciona o tópico MQTT de dispensação no horário correto.
- Recebimento e processamento dos eventos do dispositivo (taken, missed).
- Envio de notificações push ao aplicativo quando o medicamento não é retirado.
- Persistência de todos os eventos e históricos em banco de dados em nuvem.
- Exposição de endpoints REST consumidos pelo aplicativo móvel.

2.3.7 Camada de Aplicativo Móvel (Expo)

O aplicativo é desenvolvido com Expo (React Native). Funcionalidades:

- Cadastro e autenticação de pacientes e responsáveis.
- Cadastro de medicamentos com nome, dosagem, compartimento e horários.
- Visualização da programação diária de doses.

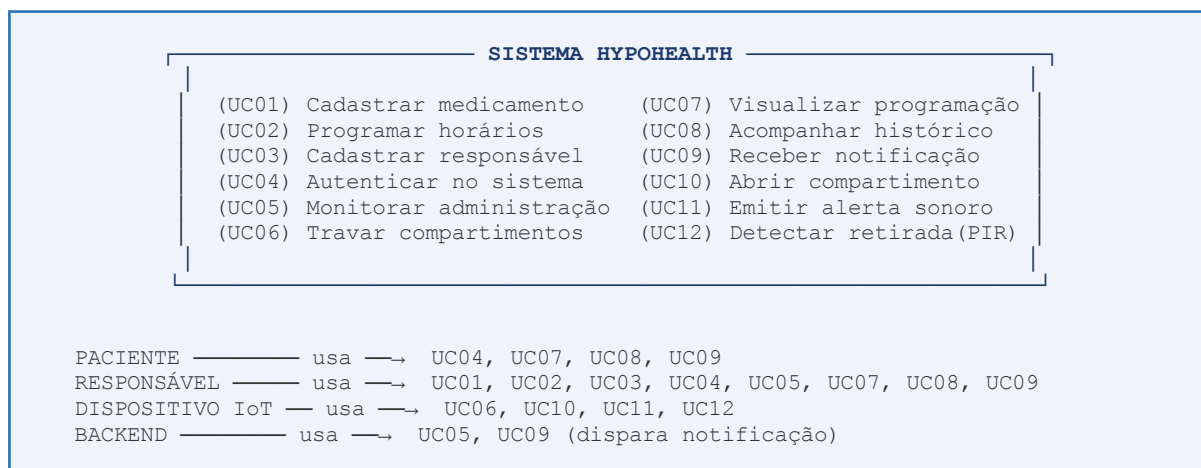
- Histórico de administrações (tomadas e perdidas).
- Recebimento de notificações push de atraso ou falha.
- Painel de monitoramento em tempo real para o responsável.

3. Casos de Uso

3.1 Atores do Sistema

Ator	Descrição
Paciente	Usuário principal. Utiliza o dispositivo físico para retirar os medicamentos e o app para visualizar programação e histórico.
Responsável	Familiar ou cuidador vinculado ao paciente. Pode cadastrar medicamentos, visualizar histórico e receber notificações de atraso.
Dispositivo IoT	Ator não humano (ESP32). Publica e consome mensagens MQTT, controla atuadores e coleta dados dos sensores.
Backend	Ator não humano (sistema). Orquestra a lógica de negócio, agenda as dispensações, processa eventos do IoT e notifica o app.

3.2 Diagrama de Casos de Uso



3.3 Descrição Detalhada dos Casos de Uso

| UC01 – Cadastrar Medicamento

Atores	Responsável
Pré-condição	Ator autenticado no sistema.
Pós-condição	Medicamento cadastrado e disponível para programação.
Fluxo Principal	1. O responsável acessa a seção de medicamentos no app. 2. Informa nome, dosagem e compartimento da caixa. 3. Confirma o cadastro. 4. O backend persiste o medicamento e sincroniza com o dispositivo.
Fluxo Alternativo	Se o compartimento já estiver ocupado, o sistema exibe aviso e solicita escolha de outro.

| UC02 – Programar Horários de Administração

Atores	Responsável
--------	-------------

Pré-condição	Medicamento cadastrado (UC01).
Pós-condição	Horários salvos e sincronizados com o dispositivo via MQTT.
Fluxo Principal	1. O responsável seleciona o medicamento. 2. Define os horários diários e/ou intervalos. 3. Configura o tempo máximo de espera após o alerta. 4. O backend armazena e envia a programação ao dispositivo via MQTT.
Fluxo Alternativo	Se houver conflito de horário/compartimento, o sistema alerta e solicita ajuste.

| UC03 – Cadastrar Responsável

Atores	Paciente ou Responsável principal
Pré-condição	Paciente autenticado.
Pós-condição	Responsável habilitado a receber notificações e acessar monitoramento.
Fluxo Principal	1. O paciente/responsável acessa configurações de conta. 2. Informa nome, e-mail e tipo de vínculo. 3. O sistema envia convite ou cria conta vinculada. 4. O responsável aceita o vínculo.
Fluxo Alternativo	Se o e-mail já estiver cadastrado, o sistema vincula automaticamente.

| UC04 – Autenticar no Sistema

Atores	Paciente, Responsável
Pré-condição	Aplicativo instalado.
Pós-condição	Usuário autenticado com sessão ativa.
Fluxo Principal	1. O usuário informa e-mail e senha no app. 2. O backend valida e retorna token JWT. 3. O app armazena o token e redireciona ao painel.
Fluxo Alternativo	Credenciais inválidas: exibir erro. Após 5 tentativas, bloquear temporariamente.

| UC05 – Monitorar Administração

Atores	Backend (automático), Responsável (visualização)
Pré-condição	Programação de horários ativa (UC02).
Pós-condição	Evento de administração registrado no histórico.
Fluxo Principal	1. No horário programado, o backend publica MQTT de dispensação. 2. O dispositivo abre o compartimento e aciona o buzzer. 3. O PIR monitora a retirada durante o intervalo configurado. 4. Medicamento retirado → ESP32 publica evento 'taken'. 5. Não retirado → ESP32 publica evento 'missed' (UC09).
Fluxo Alternativo	Falha de conexão: ESP32 armazena localmente e sincroniza ao reconectar.

| UC06 – Controlar e Travar Compartimentos

Atores	Dispositivo IoT
Pré-condição	Programação recebida via MQTT.
Pós-condição	Compartimento correto liberado; demais bloqueados.
Fluxo Principal	1. No horário programado, o driver ULN2003 recebe pulsos STEP/DIR do ESP32. 2. O motor de passos gira até o compartimento correto. 3. Apenas esse compartimento é desbloqueado. 4. Os demais permanecem travados fisicamente. 5. Após o intervalo de espera, o compartimento é travado novamente.
Fluxo Alternativo	Se o motor não atingir a posição, publicar evento de erro ao backend.

| UC07 – Visualizar Programação

Atores	Paciente, Responsável
Pré-condição	Usuário autenticado.
Pós-condição	Programação visualizada.
Fluxo Principal	1. O usuário acessa a tela de programação. 2. O app consulta a API REST e exibe medicamentos e horários. 3. O usuário pode filtrar por dia da semana ou período.
Fluxo Alternativo	Sem medicamentos cadastrados: exibir estado vazio com orientação.

| UC08 – Acompanhar Histórico

Atores	Paciente, Responsável
Pré-condição	Usuário autenticado; ao menos uma dose registrada.
Pós-condição	Histórico visualizado com indicadores de adesão.
Fluxo Principal	1. O usuário acessa a tela de histórico. 2. O app lista as administrações com status (tomado/perdido). 3. O usuário pode filtrar por data, medicamento ou status.
Fluxo Alternativo	Sem histórico: exibir mensagem informativa.

| UC09 – Notificar Responsável por Atraso

Atores	Backend (dispara), Responsável (recebe)
Pré-condição	UC05 identificou que o medicamento não foi retirado no prazo.
Pós-condição	Responsável notificado; atraso registrado.
Fluxo Principal	1. Backend recebe evento 'missed' via MQTT. 2. Registra o atraso no histórico com timestamp.

	3. Envia notificação push a todos os responsáveis vinculados. 4. O responsável visualiza a notificação no app e pode tomar ação.
Fluxo Alternativo	Dispositivo offline: notificação entregue quando ficar online novamente.

| UC10 – Abrir Compartimento (IoT)

Atores	Dispositivo IoT
Pré-condição	Comando de dispensação recebido via MQTT.
Pós-condição	Compartimento aberto e buzzer acionado.
Fluxo Principal	1. ESP32 recebe mensagem MQTT com id do compartimento. 2. Envia pulsos STEP/DIR ao driver ULN2003. 3. Motor de passos rotaciona até a posição correta. 4. Compartimento destravado e buzzer acionado.
Fluxo Alternativo	Posição não atingida: publicar evento de falha.

| UC11 – Emitir Alerta Sonoro (Buzzer)

Atores	Dispositivo IoT
Pré-condição	Compartimento aberto (UC10).
Pós-condição	Alerta sonoro emitido ao paciente.
Fluxo Principal	1. O buzzer emite sinal sonoro por duração configurada. 2. Após o alerta, o sistema aguarda a detecção pelo PIR.
Fluxo Alternativo	Alerta pode ser repetido em intervalos se configurado.

| UC12 – Detectar Retirada (Sensor PIR)

Atores	Dispositivo IoT
Pré-condição	Compartimento aberto (UC10).
Pós-condição	Evento de retirada ou atraso publicado ao backend.
Fluxo Principal	1. PIR monitora o compartimento durante o intervalo configurado. 2. Movimento detectado → publica evento 'taken' via MQTT. 3. Sem movimento no prazo → publica evento 'missed' via MQTT. 4. Compartimento é travado ao final do intervalo.
Fluxo Alternativo	Falha no sensor: publicar evento de erro e notificar backend.

4. Descrição da Comunicação Entre Elementos

4.1 Fluxo Principal: Dispensação de Medicamento

- [1] **SCHEDULER (Backend)**: Horário programado chega → publica tópico MQTT: `hypohealth/{id}/dispense`
- [2] **BROKER (HiveMQ)**: Roteia a mensagem para o ESP32 assinante do tópico
- [3] **ESP32**: Recebe o comando → envia pulsos STEP/DIR ao driver ULN2003
- [4] **DRIVER ULN2003** → **MOTOR**: Motor de passos gira e posiciona o compartimento correto
- [5] **ESP32**: Trava os demais compartimentos → aciona o buzzer
- [6] **SENSOR PIR**: Monitora o compartimento pelo intervalo configurado
- [6a] **ESP32** → **MQTT (taken)**: Medicamento retirado: ESP32 publica evento 'taken'
- [6b] **ESP32** → **MQTT (missed)**: Não retirado no prazo: ESP32 publica evento 'missed'
- [7a] **BACKEND**: Recebe 'taken' → registra administração concluída no banco
- [7b] **BACKEND**: Recebe 'missed' → registra atraso → envia push ao responsável
- [8] **APLICATIVO MÓVEL**: Exibe notificação push ao responsável e atualiza histórico

4.2 Fluxo de Cadastro e Sincronização

1. Aplicativo envia requisição POST/PUT via REST (HTTPS) ao backend.
2. Backend valida os dados, persiste no banco e retorna confirmação.
3. Backend publica a nova programação no tópico MQTT `hypohealth/{deviceId}/schedule`.
4. ESP32 recebe a programação atualizada e armazena localmente em memória flash.
5. Dispositivo opera conforme a nova programação na próxima execução.

4.3 Mecanismo de Resiliência (Offline)

- **ESP32**: armazena horários em memória flash. Em caso de queda de Wi-Fi, continua operando e faz buffer dos eventos para sincronizar ao reconectar.
- **Backend**: implementa retry para publicação MQTT e entrega de notificações push.
- **Aplicativo**: exibe dados em cache quando sem conexão.

5. Mapeamento de Requisitos

5.1 Requisitos Funcionais x Casos de Uso

RF	Requisito	Caso de Uso
RF01	Cadastro e Configuração de Medicamentos	UC01
RF02	Programação de Horários	UC02
RF03	Controle de Compartimentos (IoT)	UC06, UC10
RF04	Bloqueio de Compartimentos Indevidos	UC06
RF05	Emissão de Alerta Sonoro	UC11
RF06	Registro de Retirada de Medicamento	UC12, UC05
RF07	Monitoramento de Atraso na Administração	UC05, UC12
RF08	Notificação ao Responsável	UC09
RF09	Aplicativo Móvel para Monitoramento	UC07, UC08, UC09
RF10	Integração com Servidor em Nuvem	UC05 (MQTT)
RF11	Sincronização de Dados	UC02, UC05
RF12	Cadastro de Responsáveis	UC03

5.2 Restrições de Projeto Atendidas

RP	Restrição	Solução Adotada
RP01	Tecnologias Backend	Node.js + Express + TypeScript
RP02	Integração com IoT	ESP32 + Driver ULN2003 + Motor de Passos + PIR + Buzzer
RP03	Comunicação IoT	MQTT via HiveMQ Cloud
RP04	Aplicativo Móvel	Expo (React Native) – iOS e Android
RP05	Arquitetura Cliente-Servidor	Três camadas: IoT, Backend, Mobile
RP09	Pipeline CI/CD	GitHub Actions
RP12	Containerização	Docker

6. Cronograma e Entregas por Sprint

Sprint	Período	Entregas Planejadas
Sprint 1	13/04 – 30/04/2026	Arquitetura definida; setup repositório e Docker; backend com autenticação; protótipo IoT: ESP32 + ULN2003 + motor + buzzer + PIR; wireframes no Figma; pipeline CI básico.
Sprint 2	04/05 – 21/05/2026	Cadastro de medicamentos e horários (RF01, RF02); integração MQTT backend ↔ ESP32; controle de compartimentos com motor e driver ULN2003 (RF03–RF05); telas do app; testes automatizados.
Sprint 3	25/05 – 11/06/2026	Detecção PIR e registro de retirada (RF06, RF07); notificações push ao responsável (RF08); histórico e monitoramento no app (RF09); CD automatizado; documentação técnica final.
Final	Semana de 22/07/2026	Apresentação para banca com demonstração do sistema completo (dispositivo físico + app + backend em nuvem).